Avaliação Técnica

Desenvolvedor Front End — Angular

Avalia conhecimentos práticos em Angular 17+, RxJS, NgRx/Signals, Angular Material, testes unitários, TypeScript e integração com APIs REST.

⏱ **Prazo de entrega**	📦 **Entrega**
4 dias corridos após recebimento	Encaminhar este documento com as perguntas respondidas e com o link do código público em sua conta do GitHub

1. TypeScript e Qualidade de Código

1.1. Refatoração

Considerando seus conhecimentos de TypeScript, qualidade de código e boas práticas, quais melhorias você faria no seguinte código:

```
class Produto {
 id: any;
 descricao: any;
 quantidadeEstoque: any;

 constructor(id: any, descricao: any, quantidadeEstoque: any) {
   this.id = id;
   this.descricao = descricao;
   this.quantidadeEstoque = quantidadeEstoque;
 }
}

class Verdureira {
 produtos: any;

 constructor() {
   this.produtos = [
     new Produto(1, 'Maçã', 20),
     new Produto(2, 'Laranja', 0),
     new Produto(3, 'Limão', 20)
   ];
 }

 getDescricaoProduto(produtoId: any) {
   let produto;

   for (let index = 0; index < this.produtos.length; index++) {
     if (this.produtos[index].id == produtoId) {
       produto = this.produtos[index];
     }
   }

   return produto.id + ' - ' + produto.descricao + ' (' + produto.quantidadeEstoque + 'x)';
 }

 hasEstoqueProduto(produtoId: any) {
   let produto;

   for (let index = 0; index < this.produtos.length; index++) {
     if (this.produtos[index].id == produtoId) {
       produto = this.produtos[index];
     }
   }

   if (produto.quantidadeEstoque > 0) {
     return true;
   } else {
     return false;
   }
 }

}
```

Añade encabezados (Formato > Estilos de párrafo) y aparecerán en el índice.

1.2. Generics e tipos utilitários

Implemente uma função genérica filtrarEPaginar<T> que recebe um array, um predicado de filtro e parâmetros de paginação (página e tamanho). A função deve retornar os itens da página atual e o total de registros filtrados. Use tipagem completa — sem any.
Exemplo de parâmetros:

```
 filtrarEPaginar<T>(
  data: T[],
  filterFn: (item: T) => boolean,
  params: PaginaParams
): Pagina<T>
```

Demonstre o uso com um exemplo concreto (ex: array de usuários com filtro por nome).

2. Angular — Fundamentos e Reatividade

2.1. Change Detection e OnPush

O componente abaixo usa ChangeDetectionStrategy.OnPush, mas o nome não é exibido na tela. Identifique o problema, explique o motivo e proponha a correção — sem alterar a estratégia, sem modificar PessoaService e sem remover o setInterval.

```
import { ChangeDetectionStrategy, Component, Injectable, OnInit, OnDestroy } from '@angular/core';
import { of, Subscription } from 'rxjs';
import { delay } from 'rxjs/operators';

@Injectable()
class PessoaService {
 /** @description Mock de uma busca em API com retorno em 0.5 segundos */
 buscarPorId(id: number) {
   return of({ id, nome: 'João' }).pipe(delay(500));
 }
}

@Component({
 selector: 'app-root',
 providers: [PessoaService],
 changeDetection: ChangeDetectionStrategy.OnPush,
 template: `<h1>{{ texto }}</h1>`,
})
export class AppComponent implements OnInit, OnDestroy {
 texto: string;
 contador = 0;
 subscriptionBuscarPessoa: Subscription;
 constructor(private readonly pessoaService: PessoaService) {}

 ngOnInit(): void {
   this.subscriptionBuscarPessoa = this.pessoaService.buscarPorId(1).subscribe((pessoa) => {
     this.texto = `Nome: ${pessoa.nome}`;
   });

   setInterval(() => this.contador++, 1000);
 }

 ngOnDestroy(): void {
   /** ... */
 }
```

```
}
```

2.2. RxJS — eliminando subscriptions aninhadas

Refatore o código abaixo eliminando o subscribe dentro de subscribe. Use operadores RxJS adequados, evite memory leaks e explique brevemente sua escolha de operador:

```
ngOnInit(): void {
  const pessoaId = 1;

  this.pessoaService.buscarPorId(pessoaId).subscribe(pessoa => {
    this.pessoaService.buscarQuantidadeFamiliares(pessoaId).subscribe(qtd => {
      this.texto = `Nome: ${pessoa.nome} | familiares: ${qtd}`;
    });
  });
}
```

2.3. RxJS — busca com debounce

Implemente um campo de busca reativo em um componente Angular que:

- Aguarde 500 ms após o usuário parar de digitar antes de disparar a requisição (debounce)
- Cancele a requisição anterior caso o usuário digite novamente (evite race condition)
- Exiba um indicador de loading enquanto a requisição está em andamento
- Gerencie a subscription sem memory leak

Mostre o serviço, o componente e o template com async pipe.

2.4. Performance — OnPush e trackBy

Considere uma lista com centenas de itens renderizados com @for (ngFor). Explique:

- Por que usar trackBy melhora a performance e como implementá-lo corretamente
- Como ChangeDetectionStrategy.OnPush pode reduzir ciclos desnecessários de detecção neste cenário
- Qual seria o impacto de usar a estratégia Default neste caso

3. Gerenciamento de Estado

3.1. Angular Signals — estado local

Crie um componente de contador de itens no carrinho usando exclusivamente Signals. O componente deve expor:

- Um signal para a lista de itens
- Um computed para o total (quantidade × preço)
- Um método para adicionar e remover itens
- Um output() que emite sempre que o total mudar

3.2. Gerenciamento de Estado com NgRx (Feature To-do)

Implemente a estrutura de estado para uma lista de tarefas (To-do) utilizando os padrões recomendados do NgRx. A implementação deve incluir:

- **Actions:** Definir ações para: `loadTodos`, `loadTodosSuccess`, `loadTodosError` e `toggleTodoComplete`.
- **Reducer:** Implementar o estado inicial e a transição de estados usando `createReducer`, garantindo a tipagem forte do estado.
- **Selectors:** Criar seletores utilizando `createSelector`:
    - `selectAllTodos`: Retorna a lista completa.
    - `selectPendingTodos`: Retorna apenas as tarefas não concluídas.
- **Effect:** Criar um `Effect` que gerencie o fluxo assíncrono: ao disparar `loadTodos`, deve realizar uma chamada HTTP (mockada) e despachar a ação de sucesso ou erro conforme o resultado.

Não é necessário implementar o back-end — use HttpClient com uma URL fictícia.getProductDescription

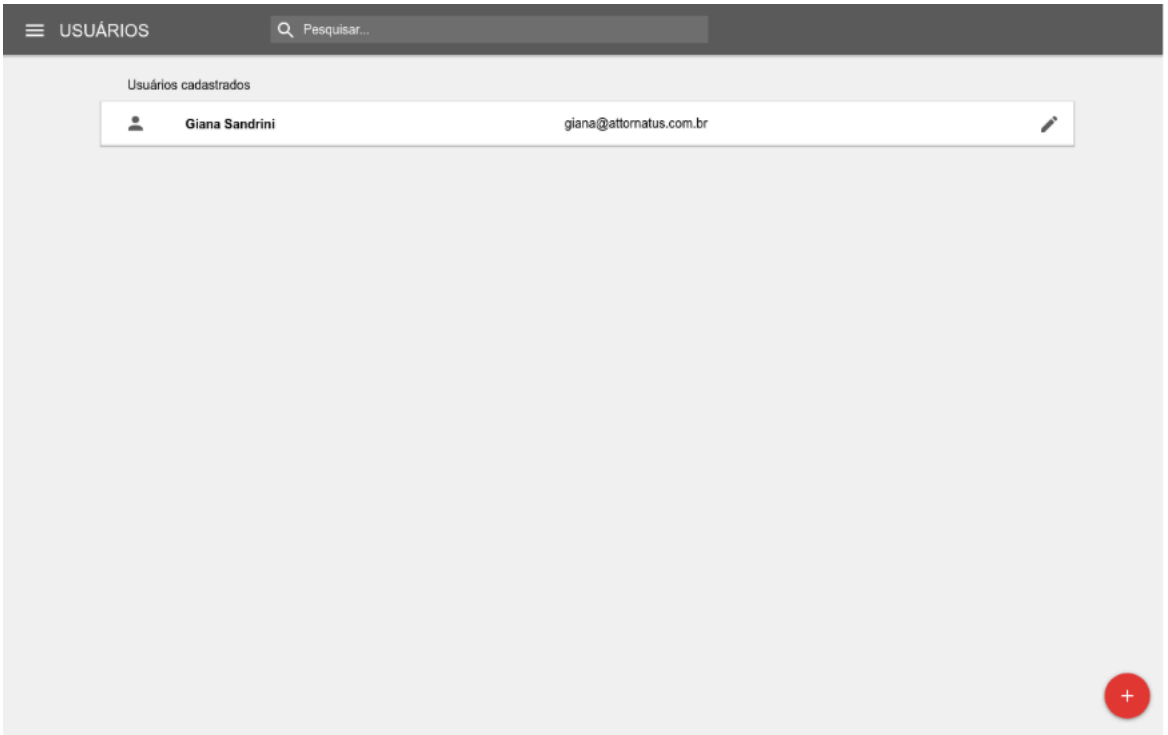
4. Desafio Prático — Aplicação Angular

Stack obrigatória: Angular 17+ · Angular Material · NgRx ou Signals · RxJS · Vitest ou Jest

4.1 O que construir

Reproduzir o protótipo de uma listagem de usuários. A listagem deverá ter as seguintes funcionalidades:

Tela de listagem de usuários:

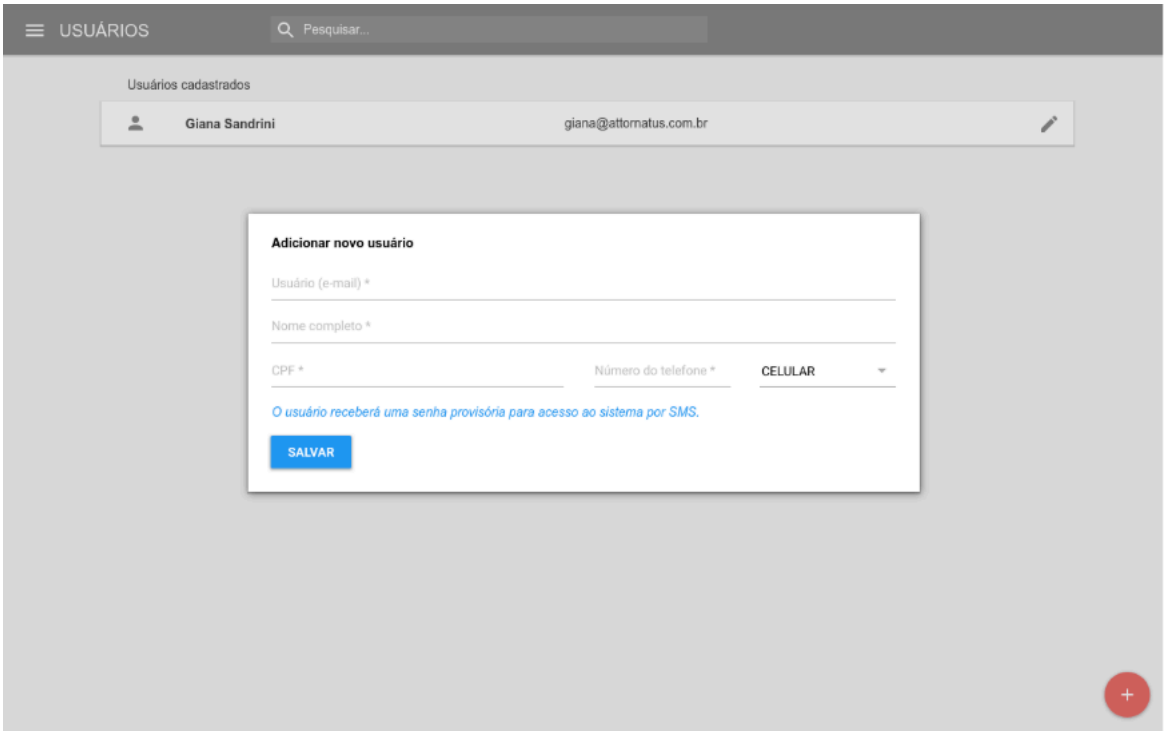


Listagem de usuários:

- Cards com os campos: nome, e-mail e botão de editar

- Filtro por nome com debounce de 300ms
- Estado de loading durante o carregamento e mensagem de erro em caso de falha
- Os dados podem vir de uma API mockada (JSON Server, MSW ou array estático em serviço)

Modal de cadastro de novo usuário com abertura através do botão vermelho que aparece na listagem:



Criação e edição de usuário (em modal):

- Formulário reativo com campos: e-mail (obrigatório), nome (obrigatório), cpf (obrigatório), telefone (obrigatório) e tipo de telefone.
- Validação com mensagens de erro por campo
- Botão de salvar desabilitado enquanto o formulário estiver inválido
- Quando for edição o formulário deve ser preenchido automaticamente

4.2 Requisitos técnicos

- Pelo menos 2 operadores RxJS além de map e tap em uso real (ex: switchMap, forkJoin, catchError, debounceTime)
- Componentes standalone
- Subscriptions gerenciadas sem memory leaks (takeUntilDestroyed, take, async pipe ou unsubscribe manual no ngOnDestroy)
- Cobertura de testes acima de 60% (Preferencialmente usando Vitest ou Jest)

4.3 Diferenciais (não obrigatórios)

- Nx Monorepo com separação em bibliotecas (ex: feature-users, data-access-users, ui)
- Paginação na listagem
- Validação de formato dos dados do formulário (e-mail,cpf e telefone)
- Melhorias no projeto em relação a tela apresentada no protótipo

---

⚠️ Inclua um README.md com instruções de instalação e execução. Projetos que não rodam localmente não serão avaliados.

✅ Dúvidas? Entre em contato pelo e-mail informado no processo seletivo. Boa sorte! 🚀

Estamos ansiosos para conhecer a sua solução.

Respostas

1.1. Refatoração

Melhorias implementadas:

- **Tipagem Estrita:** Substituição de any por interfaces e tipos específicos para garantir a segurança do código e suporte do Intellisense.
- **Padronização em Inglês:** Nomes de classes, métodos e variáveis convertidos para o inglês, seguindo as melhores práticas da indústria.
- **Uso de Métodos de Array:** Substituição de loops for manuais pelo método .find(), tornando o código mais declarativo e legível.
- **Encapsulamento:** Uso de private readonly para proteger o estado interno da classe contra modificações externas acidentais.
- **Template Literals:** Uso de ${} para construir strings, eliminando concatenações manuais confusas

```
interface IProduct {
  id: number;
  description: string;
  stockQuantity: number;
}

class Product implements IProduct {
  constructor(
    public id: number,
    public description: string,
    public stockQuantity: number
  ) {}
}

class GroceryStore {
  private readonly products: IProduct[];
  constructor() {
    this.products = [
      new Product(1, 'Apple', 20),
      new Product(2, 'Orange', 0),
      new Product(3, 'Lemon', 20)
    ];
  }


getProductDescription(productId: number): string {
    const product = this.products.find(p => p.id === productId);

    if (!product) return 'Product not found';

        return
    `${product.id} - ${product.description} (${product.stockQuantity}x)`;

  }

  hasProductInStock(productId: number): boolean {
    const product = this.products.find(p => p.id === productId);
    return !!product && product.stockQuantity > 0;
  }
}
```

1.2. Generics e tipos utilitários

Explicação técnica:

- **Generics (<T>):** Permite que a função seja reutilizável para qualquer tipo de dado (usuários, produtos, etc.) mantendo a segurança de tipos.
- **Type Safety:** Foram criadas interfaces para os parâmetros de entrada (PaginationParams) e para a estrutura de retorno (PaginationResult<T>), eliminando o uso de any.
- **Lógica de Paginação:** Utiliza filter para aplicar o predicado, slice para extrair apenas os itens da página solicitada e calcula o total com base no array filtrado.

```
interface PaginationParams {
  page: number;
  pageSize: number;
}

interface PaginationResult<T> {
  items: T[];
  total: number;
}

/**
 * Função genérica para filtrar e paginar um array de dados.
 */
function filterAndPaginate<T>(
  data: T[],
  filterFn: (item: T) => boolean,
  params: PaginationParams
): PaginationResult<T> {
  const filteredData = data.filter(filterFn);

  const total = filteredData.length;

  const startIndex = (params.page - 1) * params.pageSize;
  const endIndex = startIndex + params.pageSize;
  const items = filteredData.slice(startIndex, endIndex);

  return {
    items,
    total
  };
}

interface User {
  id: number;
  name: string;
}

const users: User[] = [
  { id: 1, name: 'Grediana' },
  { id: 2, name: 'Carlos' },
  { id: 3, name: 'Lucía' },
  { id: 4, name: 'Saymon' }
];

const params: PaginationParams = { page: 1, pageSize: 2 };

const result = filterAndPaginate<User>(
```

```
  users,
  (user) => user.name.toLowerCase().includes('a'),
  params
);

console.log(result);
// Retorno esperado: { items: [{id: 1, name: 'Grediana'}, {id: 2, name: 'Carlos'}], total: 4 }
```

2.1. Change Detection e OnPush

Identificação do Problema:

O problema ocorre porque o componente utiliza `ChangeDetectionStrategy.OnPush`. Nesta estratégia, o Angular não dispara a detecção de mudanças automaticamente quando propriedades internas são alteradas dentro de um subscribe assíncrono.

Motivo:

Com `OnPush`, a interface só é atualizada se houver uma nova referência em um `@Input`, um evento de UI ou se utilizarmos o pipe async. Como o valor de this.texto é atribuído manualmente após o delay do serviço, o Angular não é notificado de que a view precisa ser renderizada novamente.

Correção Proposta:

A correção consiste em transformar a propriedade texto em um Observable (`texto$`) e consumi-la no template via Pipe async. O pipe async gerencia a subscrição e sinaliza automaticamente ao `OnPush` que houve uma mudança, além de evitar memory leaks

```
import { ChangeDetectionStrategy, Component, Injectable, OnInit, OnDestroy } from '@angular/core';
import { of, Observable } from 'rxjs';
import { delay, map } from 'rxjs/operators';

@Injectable()
class PessoaService {
  /** @description Mock de uma busca em API com retorno em 0.5 segundos */
  buscarPorId(id: number) {
    return of({ id, nome: 'João' }).pipe(delay(500));
  }
}

@Component({
  selector: 'app-root',
  providers: [PessoaService],
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `<h1>{{ texto$ | async }}</h1>`,
})
export class AppComponent implements OnInit, OnDestroy {
  texto$: Observable<string>;
  contador = 0;

  constructor(private readonly pessoaService: PessoaService) {}

  ngOnInit(): void {
    this.texto$ = this.pessoaService.buscarPorId(1).pipe(
      map(pessoa => `Nome: ${pessoa.nome}`)
    );

    setInterval(() => this.contador++, 1000);
```

```
  }

  ngOnDestroy(): void {
  }
}
```

2.2. RxJS — eliminando subscriptions aninhadas

Explicação da escolha do operador:

A melhor abordagem é utilizar o operador **`switchMap`**. Ele permite transformar a emissão do primeiro Observable (`personById$`) no segundo fluxo (`familyMemberCount$`) de forma linear.

- **Vantagem:** O `switchMap` evita o "callback hell" de subscrições aninhadas , cancela automaticamente fluxos anteriores se houver uma nova emissão e facilita o gerenciamento de vazamentos de memória (memory leaks)

```
ngOnInit(): void {
  const personId = 1;

  this.personService.getPersonById(personId).pipe(
    switchMap(person =>
      this.personService.getFamilyMemberCount(personId).pipe(
        map(count => ({ person, count }))
      )
    ),
    takeUntilDestroyed(this.destroyRef)
  ).subscribe(({ person, count }) => {
    this.text = `Name: ${person.name} | Family Members: ${count}`;
  });
}
```

2.3. RxJS — busca com debounce

Explicação técnica:

- **`FormControl`** e **`valueChanges`**: Utilizamos um formulário reativo para observar as mudanças no campo de busca.
- **`debounceTime(500)`**: Garante que a requisição só seja disparada 500ms após o usuário parar de digitar.
- **`distinctUntilChanged()`**: Evita requisições duplicadas se o valor de busca for o mesmo (ex: digitar algo e apagar rapidamente).
- **`switchMap()`**: É o operador chave aqui. Ele cancela a subscrição anterior se uma nova emissão ocorrer, resolvendo o problema de *race condition* e cancelando requisições HTTP pendentes.
- **`Async Pipe`**: Utilizado no template para gerenciar a subscrição automaticamente, evitando *memory leaks*.

1. Service (search.service.ts)

```
export interface SearchResult {
  id: number;
  name: string;
  email: string;
}

@Injectable({ providedIn: 'root' })
export class SearchService {
  constructor(private http: HttpClient) {}

  search(term: string): Observable<SearchResult[]> {
    return
this.http.get<SearchResult[]>(`https://api.example.com/users?q=${term}`
);
  }
}
```

2. Component (search.component.ts)

```
import { Component, OnInit } from '@angular/core';
import { FormControl } from '@angular/forms';
import { Observable, of } from 'rxjs';
import { debounceTime, distinctUntilChanged, switchMap, tap, catchError, finalize } from 'rxjs/operators';

@Component({
  selector: 'app-user-search',
  templateUrl: './search.component.html'
})
export class SearchComponent implements OnInit {
  searchControl = new FormControl('', { nonNullable: true });
  results$: Observable<SearchResult[]>;
  isLoading = false;

  constructor(private searchService: SearchService) {}

  ngOnInit(): void {
    this.results$ = this.searchControl.valueChanges.pipe(
      debounceTime(500),
      distinctUntilChanged(),
      tap(() => (this.isLoading = true)),
      switchMap(term => {
        if (!term.trim()) {
          this.isLoading = false;
          return of([]);
        }
        return this.searchService.search(term).pipe(
          catchError(() => of([])),
          finalize(() => (this.isLoading = false))
        );
      })
    );
  }
}
```

3. Template (search.component.html)

```
<input type="text" [formControl]="searchControl" placeholder="Search users...">

<div *ngIf="isLoading">Loading results...</div>

<ul *ngIf="results$ | async as users">
  <li *ngFor="let user of users">
      {{ user.name }} ({{ user.email }})

  </li>
  <li *ngIf="users.length === 0 && !isLoading">No results found.</li>
</ul>
```

2.4. Performance — OnPush e trackBy

- **Por que usar `trackBy` melhora a performance e como implementá-lo corretamente:** Por padrão, quando o Angular detecta mudanças em uma lista, ele pode re-renderizar todos os elementos do DOM se a referência do array mudar, mesmo que os dados internos sejam os mesmos. O `trackBy` permite que o Angular identifique cada item de forma única (geralmente pelo `id`). Assim, o framework altera apenas os elementos que realmente sofreram mudanças, evitando manipulações desnecessárias no DOM e economizando processamento. No Angular 17, isso é feito de forma nativa na sintaxe `@for` através do parâmetro `track`.
- **Como `ChangeDetectionStrategy.OnPush` reduz ciclos desnecessários:** Com a estratégia `Default`, o Angular verifica todos os componentes da árvore sempre que qualquer evento ocorre na aplicação. Ao usar `OnPush`, o componente só entra no ciclo de verificação se uma de suas entradas (`@Input`) mudar de referência, se um evento de UI disparar nele ou se um Observable emitir um novo valor via `AsyncPipe`. Isso reduz drasticamente a carga de trabalho do navegador em listas grandes.
- **Impacto de usar a estratégia Default neste caso:** Em uma lista com centenas de itens, a estratégia `Default` causaria uma queda de performance visível (lag), pois cada pequena ação na tela forçaria o Angular a verificar cada um dos itens da lista, mesmo que nada neles tenha mudado.

```
@for (user of users; track user.id) {

  <app-user-card [user]="user"></app-user-card>

}
```

3. Gerenciamento de Estado

3.1. Angular Signals — estado local

- **`signal<Item[]>`: Utilizado para armazenar a lista de itens de forma reativa.**
- **`computed`: Define o valor total automaticamente. Ele é recalculado apenas quando o sinal de itens muda, garantindo alta performance.**
- **`effect`: Monitora as mudanças no total e dispara o `output()` de forma reativa sempre que o valor é alterado.**

- **`update()`: Método moderno dos Signals para atualizar o estado de forma imutável**

```
import { Component, signal, computed, effect, output } from '@angular/core';


interface CartItem {
  id: number;
  name: string;
  quantity: number;
  price: number;
}
@Component({
  selector: 'app-cart-counter',
  standalone: true,
  template: `
    <div>
      <h3>Items in Cart: {{ items().length }}</h3>
      <p>Total Price: {{ totalPrice() | currency }}</p>
    </div>
  `
})
export class CartCounterComponent {
  items = signal<CartItem[]>([]);
  totalPrice = computed(() =>
    this.items().reduce((acc, item) => acc + (item.quantity * item.price), 0)
  );


  totalChanged = output<number>();
  constructor() {
    effect(() => {
      this.totalChanged.emit(this.totalPrice());
    });
  }
```

```
  addItem(newItem: CartItem): void {
    this.items.update(currentItems => [...currentItems, newItem]);
  }
  removeItem(id: number): void {
    this.items.update(currentItems =>
      currentItems.filter(item => item.id !== id)
    );
  }
  updateQuantity(id: number, newQuantity: number): void {
    this.items.update(currentItems =>
      currentItems.map(item =>
        item.id === id ? { ...item, quantity: newQuantity } : item
      )
    );
  }
}
}
```

3.2. Gerenciamento de Estado com NgRx (Feature To-do)

1. Actions

```
import { createActionGroup, emptyProps, props } from '@ngrx/store';
import { Todo } from './todo.model';
export const TodoActions = createActionGroup({
  source: 'Todo Page',
  events: {
    'Load Todos': emptyProps(),
    'Load Todos Success': props<{ todos: Todo[] }>(),
    'Load Todos Error': props<{ error: string }>(),
    'Toggle Todo Complete': props<{ id: number }>(),
  }
```

```
});
```

2. Reducer

```
export interface TodoState {
  todos: Todo[];
  loading: boolean;
  error: string | null;
}
export const initialState: TodoState = {
  todos: [],
  loading: false,
  error: null
};
export const todoReducer = createReducer(
  initialState,
  on(TodoActions.loadTodos, (state) => ({ ...state, loading: true })),
  on(TodoActions.loadTodosSuccess, (state, { todos }) => ({
    ...state,
    todos,
    loading: false
  })),
  on(TodoActions.loadTodosError, (state, { error }) => ({
    ...state,
    error,
    loading: false
  })),
  on(TodoActions.toggleTodoComplete, (state, { id }) => ({
    ...state,
    todos: state.todos.map(t => t.id === id ? { ...t, completed: !t.completed } : t)
  }))
);
```

3. Selectors

```
import { createFeatureSelector, createSelector } from '@ngrx/store';

import { TodoState } from './todo.reducer';



export const selectTodoState = createFeatureSelector<TodoState>('todos');



n
```

4. Effect

```
@Injectable()

export class TodoEffects {

  loadTodos$ = createEffect(() =>

    this.actions$.pipe(

      ofType(TodoActions.loadTodos),

      switchMap(() =>

        this.http.get<Todo[]>('https://api.exemplo.com/todos').pipe(

          map(todos => TodoActions.loadTodosSuccess({ todos })),

          catchError(error => of(TodoActions.loadTodosError({ error: error.message })))

        )

      )

    )

  );



  constructor(private actions$: Actions, private http: HttpClient) {}

}
```